

SIMATS ENGINEERING

CO4 AT2 Industry Problem - Solving Task 1

NAME : Hitesh C

REG NO : 192524005

COURSE CODE : CSA1401

COURSE NAME : Compiler Design

COURSE FACULTY : AJITHA V

Q1. Smart Retail Billing System

Concepts Covered: Intermediate Languages, Declarations, Assignment Statements

Problem Statement:

A retail store billing system calculates the final payable amount based on product price, quantity purchased, and discount percentage:

*$total_cost = price * quantity$*

*$discount_value = total_cost * discount$*

$final_bill = total_cost - discount_value$

Tasks:

- (i) Write appropriate declarations for all variables with suitable data types.*
- (ii) Generate the three-address intermediate code for the given statements.*
- (iii) Represent the same using quadruples, triples, and indirect triples.*
- (iv) Identify possible optimization opportunities in the generated intermediate code.*
- (v) Generate the optimized intermediate code and justify the improvements.*

Answer:

(i) Variable Declarations with Data Types:

float price;	; Unit price of the product
int quantity;	; Number of units purchased
float discount;	; Discount rate (e.g., 0.10 for 10% discount)

```
float total_cost;    ; Computed subtotal before discount
float discount_value; ; Calculated discount amount
float final_bill;    ; Net payable amount after discount
```

(ii) Three-Address Intermediate Code (TAC):

```
t1 = price * quantity      ; Subtotal calculation
total_cost = t1
t2 = total_cost * discount  ; Discount amount calculation
discount_value = t2
t3 = total_cost - discount_value; Net payable bill calculation
final_bill = t3
```

(iii) Intermediate Code Representations:

1. Quadruples Representation (op, arg1, arg2, result):

Index	op	arg1	arg2	result
(0)	*	price	quantity	t1
(1)	=	t1	—	total_cost
(2)	*	total_cost	discount	t2
(3)	=	t2	—	discount_value
(4)	-	total_cost	discount_value	t3
(5)	=	t3	—	final_bill

2. Triples Representation (index, op, arg1, arg2):

Index	op	arg1	arg2
(0)	*	price	quantity

(1)	=	total_cost	(0)
(2)	*	(0)	discount
(3)	=	discount_value	(2)
(4)	-	(0)	(2)
(5)	=	final_bill	(4)

3. Indirect Triples Representation:

Statement List Array:

Statement Index	Triple Pointer
statement[0]	(0)
statement[1]	(1)
statement[2]	(2)
statement[3]	(3)
statement[4]	(4)
statement[5]	(5)

(iv) Identification of Optimization Opportunities:

- Copy Propagation & Temporary Elimination: The temporary variables t1, t2, and t3 immediately transfer values to total_cost, discount_value, and final_bill respectively. These intermediate temporaries can be completely eliminated.
- Common Subexpression Elimination: After total_cost is computed, its value can be directly referenced in subsequent operations without redundant register reloads.
- Algebraic Strength Reduction: The net payable calculation can be simplified algebraically if discount factor (1 - discount) is precomputed.

(v) Optimized Intermediate Code & Justification:

```
total_cost = price * quantity
```

```
discount_value = total_cost * discount
final_bill = total_cost - discount_value
```

Justification of Improvements:

1. Instruction Count Reduction: Reduced total TAC instructions from 6 down to 3.
2. Memory & Register Overhead: Saved 3 temporary register allocations (t1, t2, t3), drastically reducing register pressure.
3. Execution Efficiency: Direct store operations eliminate redundant copy instructions, resulting in faster machine code generation.

Q2. University Scholarship Eligibility System

Concepts Covered: Boolean Expressions, Backpatching

Problem Statement:

A student is eligible for a scholarship if:

CGPA > 8.5 AND (Family Income < 300000 OR Sports Quota = TRUE)

Tasks:

- (i) Formulate the Boolean expression representing the eligibility criteria.
- (ii) Generate intermediate code using short-circuit evaluation.
- (iii) Apply backpatching to resolve forward jumps.
- (iv) Construct the final labelled intermediate code.
- (v) Draw the corresponding control-flow graph.

Answer:

(i) Boolean Expression Formulation:

```
E = (CGPA > 8.5) && (FamilyIncome < 300000 || SportsQuota == TRUE)
```

(ii) & (iii) Intermediate Code Generation & Backpatching Execution:

Let B1 = (CGPA > 8.5), B2 = (FamilyIncome < 300000), B3 = (SportsQuota == TRUE), B23 = B2 || B3, and B = B1 && B23.

Step 1: Evaluate B1 = (CGPA > 8.5) at Quad 100:

```
100: if CGPA > 8.5 goto _
101: goto _
```

Attributes: B1.truelist = [100], B1.falselist = [101]

Step 2: AND Marker M1 after B1 (M1.quad = 102):

backpatch(B1.truelist, 102) -> Instruction 100 becomes '100: if CGPA > 8.5 goto 102'.

Step 3: Evaluate B2 = (FamilyIncome < 300000) at Quad 102:

```
102: if FamilyIncome < 300000 goto _  
103: goto _
```

Attributes: B2.truelist = [102], B2.falselist = [103]

Step 4: OR Marker M2 after B2 (M2.quad = 104):

backpatch(B2.falselist, 104) -> Instruction 103 becomes '103: goto 104'.

Step 5: Evaluate B3 = (SportsQuota == TRUE) at Quad 104:

```
104: if SportsQuota == TRUE goto _  
105: goto _
```

Attributes: B3.truelist = [104], B3.falselist = [105]

Step 6: Synthesize B23 = B2 || B3 and Complete Expression B = B1 && B23:

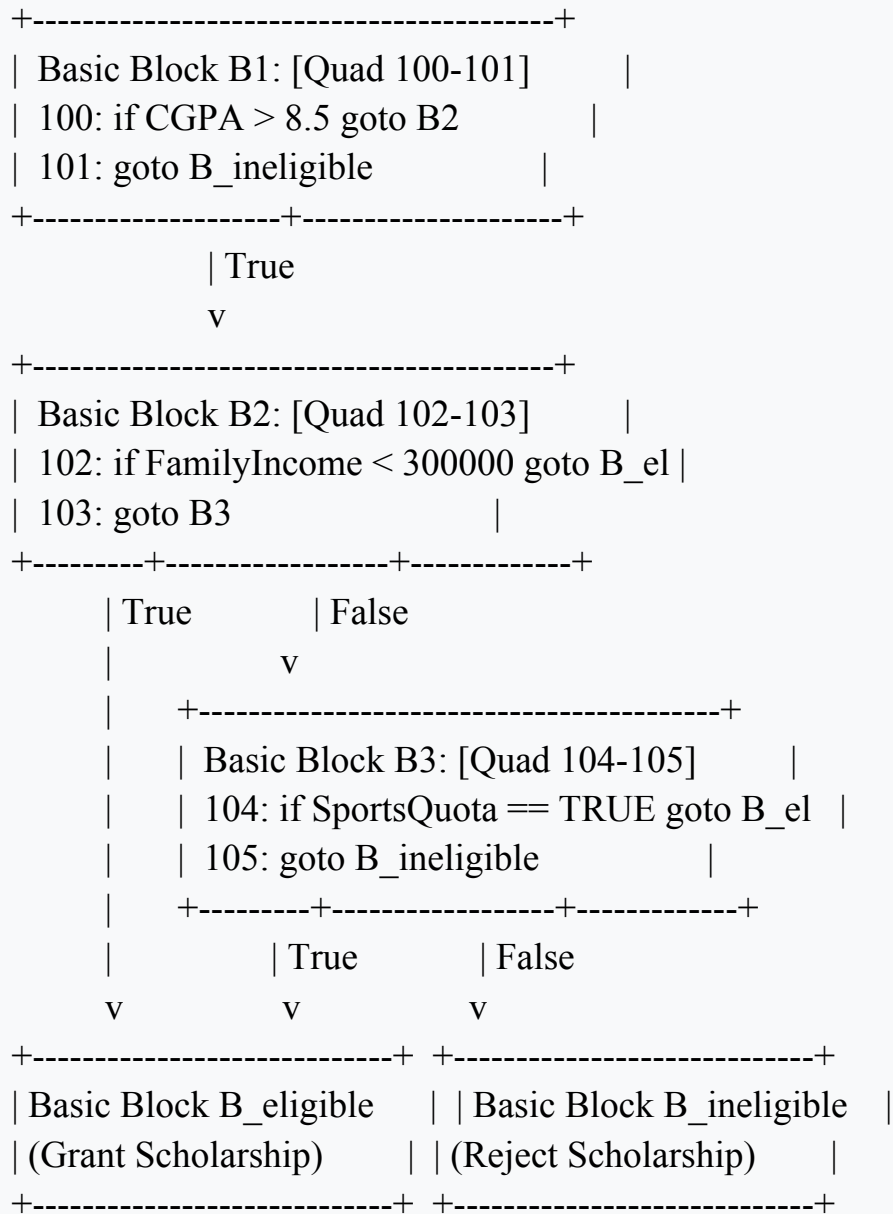
- B23.truelist = merge(B2.truelist, B3.truelist) = [102, 104]
- B23.falselist = B3.falselist = [105]
- B.truelist = B23.truelist = [102, 104]
- B.falselist = merge(B1.falselist, B23.falselist) = merge([101], [105]) = [101, 105]

(iv) Final Labelled Intermediate Code:

```
100: if CGPA > 8.5 goto 102      ; Primary eligibility test  
101: goto L_ineligible          ; Fail CGPA: reject scholarship  
102: if FamilyIncome < 300000 goto L_eligible ; Income test pass: grant  
scholarship  
103: goto 104                  ; Income fail: check sports quota  
104: if SportsQuota == TRUE goto L_eligible  ; Sports test pass: grant  
scholarship  
105: goto L_ineligible          ; All conditions failed: reject
```

106: L_eligible: ...	; Grant scholarship code
107: L_ineligible: ...	; Reject scholarship code

(v) Control-Flow Graph (CFG) Structure:



Q3. Smart Parking Management System

Concepts Covered: Case Statements, Intermediate Languages

Problem Statement:

A parking management system assigns parking zones based on vehicle type:
switch(vehicle_type)

```
{  
  case 1: TWO_WHEELER;  
  case 2: CAR;  
  case 3: BUS;  
  case 4: EMERGENCY_VEHICLE;  
}
```

Tasks:

- (i) Convert the switch statement into three-address intermediate code using conditional jumps.*
- (ii) Generate an equivalent jump-table implementation.*
- (iii) Represent the intermediate code using quadruples.*
- (iv) Compare conditional branching and jump-table implementations.*
- (v) Recommend the best approach for a large-scale parking system with justification.*

Answer:

(i) Three-Address Code using Conditional Jumps (Linear Cascade):

```
100: if vehicle_type == 1 goto L_TWO_WHEELER  
101: if vehicle_type == 2 goto L_CAR  
102: if vehicle_type == 3 goto L_BUS  
103: if vehicle_type == 4 goto L_EMERGENCY  
104: goto L_DEFAULT  
105: L_TWO_WHEELER: ... ; Process TWO_WHEELER parking  
    goto L_END  
106: L_CAR:      ... ; Process CAR parking  
    goto L_END  
107: L_BUS:      ... ; Process BUS parking  
    goto L_END  
108: L_EMERGENCY: ... ; Process EMERGENCY_VEHICLE parking  
    goto L_END  
109: L_DEFAULT:  ... ; Process Default/Invalid type  
110: L_END:
```

(ii) Jump-Table Implementation:

Jump Table Initialization:

JUMP_TABLE[0] = L_TWO_WHEELER

JUMP_TABLE[1] = L_CAR

JUMP_TABLE[2] = L_BUS

JUMP_TABLE[3] = L_EMERGENCY

```
100: if vehicle_type < 1 goto L_DEFAULT ; Bounds check lower limit
101: if vehicle_type > 4 goto L_DEFAULT ; Bounds check upper limit
102: t1 = vehicle_type - 1 ; Normalize 1-based index to 0-based offset
103: t2 = t1 * 4 ; Compute byte offset (4 bytes per pointer)
104: t3 = JUMP_TABLE[t2] ; Load target jump address from table
105: goto t3 ; Indirect jump to target zone handler
```

(iii) Quadruples Representation for Jump-Table Implementation:

Index	op	arg1	arg2	result
(0)	<	vehicle_type	1	t_cond1
(1)	if_true	t_cond1	—	L_DEFAULT
(2)	>	vehicle_type	4	t_cond2
(3)	if_true	t_cond2	—	L_DEFAULT
(4)	-	vehicle_type	1	t1
(5)	*	t1	4	t2
(6)	=[]	JUMP_TABLE	t2	t3
(7)	goto	t3	—	—

(iv) Comparison: Conditional Branching vs. Jump-Table Implementation:

Evaluation Criteria	Conditional Branching (if-else chain)	Jump-Table Implementation
Time Complexity	$O(N)$ - Linear search through cases	$O(1)$ - Constant time direct lookup
Branch Predictor Impact	High penalty due to sequential comparisons	Minimal penalty (single indirect branch)
Memory Footprint	Minimal (only branch instructions)	Requires memory array for pointer table
Scalability with Cases	Degrades as number of cases increases	Extremely scalable for dense case ranges
Best Use Case	Sparse or small case sets ($N \leq 3$)	Dense, sequential case ranges ($N \geq 4$)

(v) Recommendation for Large-Scale Parking System & Justification:

- Recommendation: Jump-Table Implementation.
- Technical Justification:
 1. Deterministic $O(1)$ Performance: Large-scale parking systems process thousands of vehicle entries per minute. A jump table guarantees instant $O(1)$ dispatch regardless of the number of vehicle categories.
 2. Hardware Pipeline Optimization: Avoids instruction pipeline stalls caused by mispredicted conditional jumps in long if-else chains.
 3. Dense Case Layout: Vehicle types are represented by consecutive integers (1, 2, 3, 4), making jump-table indexing perfectly optimal with zero array slot waste.

Q4. Online Food Delivery Processing System

Concepts Covered: Procedure Calls, Activation Records, Intermediate Code

Problem Statement:

An online food delivery platform executes the following function calls:

amount = calculateBill(order)

applyCoupon(amount)

generateInvoice(amount)

Tasks:

- (i) Generate intermediate code for the procedure calls.
- (ii) Illustrate parameter passing and return value handling.
- (iii) Design the activation record (stack frame) for each procedure call.
- (iv) Show the sequence of call and return operations using stack diagrams.
- (v) Explain how local variables, parameters, and return addresses are stored in the activation record.

Answer:

(i) Intermediate Code for Procedure Calls:

```
100: param order          ; Pass argument 'order'
101: amount = call calculateBill, 1 ; Call calculateBill with 1 argument, receive
amount
102: param amount          ; Pass argument 'amount'
103: call applyCoupon, 1    ; Call applyCoupon with 1 argument
104: param amount          ; Pass argument 'amount'
105: call generateInvoice, 1 ; Call generateInvoice with 1 argument
```

(ii) Parameter Passing and Return Value Handling:

- Parameter Passing: The caller pushes function arguments ('order', 'amount') onto the runtime stack in right-to-left order (or places them in specified ABI registers like r0, r1) before executing the call instruction.
- Return Value Handling: For non-void functions like calculateBill(), the callee stores its return result in a designated hardware register (e.g., EAX / R0). The assignment 'amount = call calculateBill, 1' moves the value from this register into the variable 'amount'.

(iii) Activation Record (Stack Frame) Structure Design:

Relative Offset	Stack Frame Component	Description & Purpose
FP + 8 + 4*n	Actual Parameters	Arguments passed from caller (e.g., order, amount)

FP + 4	Return Address	Saved Instruction Pointer (PC) to resume caller after return
FP + 0	Control Link (Saved FP)	Dynamic link pointing to caller's frame pointer
FP - 4	Saved Registers	Preserved CPU register values required by caller ABI
FP - 8 to FP - k	Local Variables	Temporaries, local variables, and internal buffers

(iv) Sequence of Call and Return Operations (Runtime Stack Trace):

Step 1: main() calls calculateBill(order)

[Push order] -> [Push Return Address RA1] -> [Push Saved FP] -> [Allocate Local Frame]

calculateBill executes -> places computed total in Return Register EAX -> unwinds frame -> returns to RA1.

Step 2: main() calls applyCoupon(amount)

[Push amount] -> [Push Return Address RA2] -> [Push Saved FP] -> [Allocate Local Frame]

applyCoupon executes -> modifies discount rates -> unwinds frame -> returns to RA2.

Step 3: main() calls generateInvoice(amount)

[Push amount] -> [Push Return Address RA3] -> [Push Saved FP] -> [Allocate Local Frame]

generateInvoice executes -> prints bill -> unwinds frame -> returns to RA3.

(v) Detailed Explanation of Activation Record Storage:

1. Parameters: Pushed by the caller prior to procedure entry. Accessible via positive offsets from the frame pointer (FP + 8, FP + 12).
2. Return Address: Automatically pushed onto stack by hardware 'call' instruction. Ensures execution resumes at the exact instruction immediately following the call.
3. Control Link (Dynamic Link): Stores caller's frame pointer at FP + 0. Restores stack boundaries upon return.
4. Local Variables & Temporaries: Allocated at negative offsets relative to FP (FP - 4, FP - 8). Deallocated instantly upon return by restoring the stack pointer (SP = FP).

Q5. Smart Industrial Safety Monitoring System

Concepts Covered: Assignment Statements, Boolean Expressions, Backpatching, Procedure Calls

Problem Statement:

An industrial safety system activates an emergency alarm if:

Temperature > 100 AND (Pressure > 80 OR GasLeak = TRUE)

When the condition is satisfied, the system invokes:

activateAlarm()

sendNotification()

Tasks:

- (i) Write the Boolean expression representing the safety condition.
- (ii) Generate intermediate code using short-circuit evaluation.
- (iii) Apply backpatching to resolve all jump targets.
- (iv) Generate intermediate code for the procedure calls executed when the condition is true.
- (v) Represent the complete control flow using labelled instructions and explain the execution sequence.

Answer:

(i) Boolean Expression Formulation:

```
E = (Temperature > 100) && (Pressure > 80 || GasLeak == TRUE)
```

(ii) & (iii) Intermediate Code Generation & Backpatching Resolution:

Let $B1 = (Temperature > 100)$, $B2 = (Pressure > 80)$, $B3 = (GasLeak == TRUE)$,
 $B23 = B2 \parallel B3$, $B = B1 \ \&\& \ B23$.

Step-by-step Backpatching Trace:

1. $B1 = (Temperature > 100)$ at Quad 100: $B1.truelist = [100]$, $B1.falselist = [101]$.
2. AND Marker M1 at Quad 102: $backpatch(B1.truelist, 102) \rightarrow$ Quad 100 becomes 'if Temperature > 100 goto 102'.
3. $B2 = (Pressure > 80)$ at Quad 102: $B2.truelist = [102]$, $B2.falselist = [103]$.
4. OR Marker M2 at Quad 104: $backpatch(B2.falselist, 104) \rightarrow$ Quad 103 becomes 'goto 104'.
5. $B3 = (GasLeak == TRUE)$ at Quad 104: $B3.truelist = [104]$, $B3.falselist = [105]$.
6. $B23.truelist = merge(B2.truelist, B3.truelist) = [102, 104]$; $B23.falselist = [105]$.
7. Overall $B.truelist = [102, 104]$; $B.falselist = merge([101], [105]) = [101, 105]$.
8. Target Binding: $backpatch(B.truelist, L_alarm)$ and $backpatch(B.falselist, L_safe)$.

(iv) Intermediate Code for Alarm Procedure Calls:

```
L_alarm:
call activateAlarm, 0    ; Invoke alarm procedure with 0 arguments
call sendNotification, 0 ; Invoke notification procedure with 0 arguments
goto L_done
```

(v) Complete Control Flow & Labelled Execution Sequence:

```
100: if Temperature > 100 goto 102 ; Primary safety check
101: goto L_safe                    ; Temperature normal: skip alarm
102: if Pressure > 80 goto L_alarm  ; Secondary pressure check
103: goto 104                      ; Pressure normal: check gas leak
104: if GasLeak == TRUE goto L_alarm ; Gas leak check
105: goto L_safe                    ; All secondary checks normal: skip alarm
106: L_alarm:                      ; Alarm Execution Handler
107: call activateAlarm, 0          ; Activate emergency siren
108: call sendNotification, 0       ; Dispatch emergency alerts
109: goto L_done
```

110: L_safe: ; Normal System Operation

111: ; Continue routine monitoring

112: L_done:

Execution Sequence Explanation:

1. Normal Temperature (≤ 100): Quad 100 branches directly to 101 -> L_safe.

Alarm routines are completely bypassed.

2. High Temperature & High Pressure (> 100 & > 80): Quad 100 -> 102 ->

L_alarm. Emergency procedures activate immediately.

3. High Temperature & Gas Leak (> 100 & GasLeak=TRUE): Quad 100 -> 102 -> 103 -> 104 -> L_alarm. Emergency procedures activate immediately.

4. High Temperature only (Pressure ≤ 80 & GasLeak=FALSE): Quad 100 -> 102 -> 103 -> 104 -> 105 -> L_safe. No alarm activated.